Streamer Functions

Prerequisites

Connecting to the WebSocket for market and portfolio updates is streamlined through two primary Feeder functions. Both functions are designed to simplify the process of subscribing to essential data streams, ensuring users have quick and easy access to the information they need.

You need to have the SDK installed for the specific language you are using. For detailed installation instructions and repository links, refer to the Installing the Upstox SDK guide.

MarketDataStreamerV3

The MarketDataStreamerV3 interface is designed for effortless connection to the market WebSocket, enabling users to receive instantaneous updates on various instruments. The following example demonstrates how to quickly set up and start receiving market updates for selected instrument keys:

Python SDK | Node.js SDK | Java SDK | PHP SDK

```
import upstox_client

def on_message(message):
    print(message)


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration), ["NSE_INDEX|Nifty 50",
"NSE_INDEX|Nifty Bank"], "full")

    streamer.on("message", on_message)
```

```
    streamer.connect()


if __name__ == "__main__":
```

In this example, you first authenticate using an access token, then instantiate MarketDataStreamerV3 with specific instrument keys and a subscription mode. Upon connecting, the streamer listens for market updates, which are logged to the console as they arrive.

Feel free to adjust the access token placeholder and any other specifics to better fit your actual implementation or usage scenario.

Exploring the MarketDataStreamerV3 Functionality

Modes

- **ltpc**: ltpc provides information solely about the most recent trade, encompassing details such as the last trade price, time of the last trade, quantity traded, and the closing price from the previous day.
- **full**: The full option offers comprehensive information, including the latest trade prices, D5 depth, 1-minute, 30-minute, and daily candlestick data, along with some additional details.
- **option_greeks**: Contains only option greeks.
- **full_d30**: full_d30 includes Full mode data plus 30 market level quotes. Plus

Functions

1. **constructor MarketDataStreamerV3(apiClient, instrumentKeys, mode)**: Initializes the streamer with optional instrument keys and mode (`full`, `ltpc`, `full_d30`, or `option_greeks`).
2. **connect()**: Establishes the WebSocket connection.
3. **subscribe(instrumentKeys, mode)**: Subscribes to updates for given instrument keys in the specified mode. Both parameters are mandatory.
4. **unsubscribe(instrumentKeys)**: Stops updates for the specified instrument keys.
5. **changeMode(instrumentKeys, mode)**: Switches the mode for already subscribed instrument keys.
6. **disconnect()**: Ends the active WebSocket connection.
7. **auto_reconnect(enable, interval, retryCount)**: Customizes auto-reconnect functionality. Parameters include a flag to enable/disable it, the interval(in seconds)

between attempts, and the maximum number of retries.

Events

- **open**: Emitted upon successful connection establishment.
- **close**: Indicates the WebSocket connection has been closed.
- **message**: Delivers market updates.
- **error**: Signals an error has occurred.
- **reconnecting**: Announced when a reconnect attempt is initiated.
- **autoReconnectStopped**: Informs when auto-reconnect efforts have ceased after exhausting the retry count.

The following documentation includes examples to illustrate the usage of these functions and events, providing a practical understanding of how to interact with the MarketDataStreamerV3 effectively.

Subscribing to Market Data on Connection Open with MarketDataStreamerV3

Python SDK | Node.js SDK | Java SDK | PHP SDK

```python
import upstox_client

def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration))

    def on_open():
        streamer.subscribe(
            ["NSE_EQ|INE020B01018", "NSE_EQ|INE467B01029"], "full")

    def on_message(message):
        print(message)

    streamer.on("open", on_open)
    streamer.on("message", on_message)

    streamer.connect()
```

Subscribing to Instruments with Delays

Python SDK Node.js SDK Java SDK PHP SDK

```
import upstox_client
import time


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration))

    def on_open():
        streamer.subscribe(
            ["NSE_EQ|INE020B01018"], "full")

    # Handle incoming market data messages\
    def on_message(message):
        print(message)

    streamer.on("open", on_open)
    streamer.on("message", on_message)
```

Subscribing and Unsubscribing Instruments

Python SDK Node.js SDK Java SDK PHP SDK

```
import upstox_client
import time


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
```

```
        upstox_client.ApiClient(configuration))

    def on_open():
        print("Connected. Subscribing to instrument keys.")
        streamer.subscribe(
            ["NSE_EQ|INE020B01018", "NSE_EQ|INE467B01029"], "full")

    # Handle incoming market data messages\
    def on_message(message):
        print(message)

    streamer.on("open", on_open)
```

Subscribe, Change Mode and Unsubscribe

Python SDK | Node.js SDK | Java SDK | PHP SDK

```
import upstox_client
import time

def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration))

    def on_open():
        print("Connected. Subscribing to instrument keys.")
        streamer.subscribe(
            ["NSE_EQ|INE020B01018", "NSE_EQ|INE467B01029"], "full")

    # Handle incoming market data messages\
    def on_message(message):
        print(message)

    streamer.on("open", on_open)
    streamer.on("message", on_message)
```

Disable Auto-Reconnect

Python SDK Node.js SDK Java SDK PHP SDK

```python
import upstox_client
import time


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration))

    def on_reconnection_halt(message):
        print(message)

    streamer.on("autoReconnectStopped", on_reconnection_halt)

    # Disable auto-reconnect feature
    streamer.auto_reconnect(False)

    streamer.connect()
```

Modify Auto-Reconnect parameters

Python SDK Node.js SDK Java SDK PHP SDK

```python
import upstox_client


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.MarketDataStreamerV3(
        upstox_client.ApiClient(configuration))

    # Modify auto-reconnect parameters: enable it, set interval to 10 seconds,
and retry count to 3
```

```
    streamer.auto_reconnect(True, 10, 3)

    streamer.connect()


if __name__ == "__main__":
    main()
```

PortfolioDataStreamer

Connecting to the Portfolio WebSocket for real-time order updates is straightforward with the PortfolioDataStreamer function. Below is a concise guide to get you started on receiving updates. For detailed API documentation, refer to the Portfolio Stream Feed API.

Python SDK | Node.js SDK | Java SDK | PHP SDK

```
import upstox_client

def on_message(message):
    print(message)


def main():
    configuration = upstox_client.Configuration()
    access_token = <ACCESS_TOKEN>
    configuration.access_token = access_token

    streamer = upstox_client.PortfolioDataStreamer(
        upstox_client.ApiClient(configuration))

    streamer.on("message", on_message)

    streamer.connect()


if __name__ == "__main__":
    main()
```

Position, Holding, and GTT order updates can be enabled by setting the corresponding flag to `True` in the constructor of the `PortfolioDataStreamer` class.

Python SDK Node.js SDK Java SDK PHP SDK

```
import upstox_client
import data_token


def on_message(message):
    print(message)


def on_open():
    print("connection opened")


def main():
    configuration = upstox_client.Configuration()
    configuration.access_token = <ACCESS_TOKEN>

    streamer =
upstox_client.PortfolioDataStreamer(upstox_client.ApiClient(configuration),
                                                        order_update=True,
                                                        position_update=True,
                                                        holding_update=True,
                                                        gtt_update=True)
```

Exploring the PortfolioDataStreamer Functionality

Constructor Parameters

1. **api_client**: Your API client instance
2. **order_update**: Set to `True` to receive real-time order updates (default: `True`)
3. **position_update**: Set to `True` to receive position updates (default: `False`)
4. **holding_update**: Set to `True` to receive holding updates (default: `False`)
5. **gtt_update**: Set to `True` to receive GTT order updates (default: `False`)

Functions

1. **constructor PortfolioDataStreamer()**: Initializes the streamer.
2. **connect()**: Establishes the WebSocket connection.
3. **disconnect()**: Ends the active WebSocket connection.
4. **auto_reconnect(enable, interval, retryCount)**: Customizes auto-reconnect functionality. Parameters include a flag to enable/disable it, the interval(in seconds)

between attempts, and the maximum number of retries.

Events

- **open**: Emitted upon successful connection establishment.
- **close**: Indicates the WebSocket connection has been closed.
- **message**: Delivers market updates.
- **error**: Signals an error has occurred.
- **reconnecting**: Announced when a reconnect attempt is initiated.
- **autoReconnectStopped**: Informs when auto-reconnect efforts have ceased after exhausting the retry count.